KidsCode (Kids Code Academy)

Project Definition Document

Course: BIL 481 – Software Engineering

Assignment: Project Assignment 1

Contributing authors: Arjen Tüysüz and Mohamed Ahmed Ledib

Finalization date: November 1, 2026

Requirements baseline: Kids Code Academy – Requirements v0.14

Table of Contents

1. Project Summary
2. Objectives
3. Scope
4. Target Audience
5. Key Features
6. Deliverables
7. Resources and Budget
8. Risks and Mitigations
9. Success Criteria
10. Document Contribution Matrix

1. Project Summary

KidsCode is a multilingual web platform that teaches programming to K–12 learners through interactive Blockly and Python puzzles. The curriculum follows Course → Lesson → Level → Stop, where a Stop is either a puzzle or an informational activity. The first deployment is Arabic-first in Mauritania, with French at launch and English and Turkish planned as translations become available.

The v0.14 baseline defines a multi-organization platform with strict data isolation, scoped roles, content publishing workflows, append-only learner attempts, and low-bandwidth performance targets. The system emphasizes children's privacy, simple learning navigation, and management without requiring administrators to edit code or identifiers.

2. Objectives

Deliver a working, secure web learning experience supporting Blockly and Python; enforce organization isolation and scoped authorization; support multilingual LTR/RTL interfaces; track attempts and derive best scores; offer parent/teacher oversight and authoring/publishing workflows; and evaluate quality using repeatable automated and performance tests.

3. Scope

In scope (P0/P1): authentication and account setup; scoped permissions; tenant-aware data; curriculum hierarchy; Blockly/Python puzzle execution; grading and append-only attempt history; Continue/course-map navigation; dashboards; class administration; draft/preview/publish content; translations; CDN-backed assets; and testing/deployment.

Out of scope for initial rollout: P2 classroom sign-in with picture passwords, P2 offline app-shell support, native mobile apps, live video teaching, robotics, AI-generated lessons, and an unrestricted programming playground.

4. Target Audience

Children learning programming; parents following their children's progress; teachers managing classes; organization administrators; content authors and translators; and platform administrators operating the service.

5. Key Features

Class management: Organization admins create classes, assign teachers, set the course and content language, and manage membership.

Secure access: Scoped roles; one-time parent setup links or QR codes; HttpOnly session cookies.

Puzzle workspace: Blockly and Python solutions to curriculum-linked programming puzzles.

Learning records: Append-only attempts retain submitted programs and scores; best score is calculated from attempts.

Student navigation: A Continue button and a course map guide children to the next activity.

Content publishing: Draft, preview, publish and version workflows with immutable published content.

Localization: Independent interface/content language; correct RTL/LTR and bidirectional text.

Progress dashboards: Parents and teachers view progress only for the children in their permitted relationships or classes.

6. Deliverables

Deployed browser-based prototype; versioned source repository; secure API and organization-aware data model; Blockly/Python learning workspace; attempt/evaluation service; learner, parent, teacher and admin interfaces; content authoring and publishing flow; localization assets; automated test evidence; and project documentation.

7. Resources and Budget

Resource	Use / estimated direct cost
Two developers	300 planned person-hours total (150 each)
Existing laptops, browsers	No new hardware purchase planned
TypeScript, Blockly, Pyodide, GitHub	Open-source / existing access
Cloudflare Workers, D1, CDN/object storage	Free tiers where available; paid usage if limits exceeded
Testing tools	Open-source tools and browser developer tools

Direct cash budget is not yet approved; cloud charges are usage-dependent. Person-hours are effort estimates, not actual logged hours.

8. Risks and Mitigations

Risk	Mitigation
Cross-organization data leakage	Centralize authorization and run isolation tests on each protected endpoint
Unsafe game or content execution	Sandbox game/info content; ship game-type code only through tested CI
Slow loading on low-end phones	Lazy-load runtimes; monitor 500 KB / 20-request budget excluding Python
RTL/LTR errors	Test Arabic/French UI, mixed-direction identifiers and editor direction

Schedule overruns	Prioritize P0, then P1; defer P2; review work weekly
Lost progress or content regressions	Append-only attempts, versioned publishing, database backups and restore testing

9. Success Criteria

SC-01: 100% of selected P0 acceptance tests pass before P1 rollout evaluation.

SC-02: All protected endpoints pass automated cross-organization access tests with zero unauthorized reads or writes.

SC-03: First puzzle visit transfers at most 500 KB across at most 20 requests, excluding the Python runtime.

SC-04: Child-facing API response p95 is at most 300 ms and admin response p95 at most 1 second.

SC-05: All predefined reference puzzle solutions produce expected evaluation results; historical attempts remain intact after content edits.

10. Document Contribution Matrix

Section / task	Responsible (reviewer)
Summary, objectives, scope	Mohamed Ahmed Ledib (reviewer: Arjen Tüysüz)
Audience, features, deliverables	Arjen Tüysüz (reviewer: Mohamed Ahmed Ledib)
Resources, risks, success criteria	Mohamed Ahmed Ledib (reviewer: Arjen Tüysüz)
Editing, verification and final formatting	Arjen Tüysüz (reviewer: Mohamed Ahmed Ledib)

Each section had one responsible author and one reviewer; individual edits and commits remain traceable in GitHub.